

Object Oriented Programming - GROUP No. 18

Name : Amay Somani

Id No. : 2024ADPS0869G

Group Title : Learning Management System

My Title : Search and Filter System

My Title Description :

1. Introduction

The Search and Filter System serves as a critical navigation and discovery component within a comprehensive Learning Management System (LMS). This module empowers diverse user groups—Students, Instructors, Administrators, and Teaching Assistants—to efficiently locate and access educational resources, user profiles, and administrative content across the platform's extensive data ecosystem.

1.1 Module Purpose

The primary objective of this system is to transform the user experience from passive navigation to active, intelligent discovery. By implementing a centralized search architecture combined with sophisticated filtering mechanisms, the module reduces the time required to locate relevant content while improving the overall usability and accessibility of the LMS platform.

1.2 Core Capabilities

The system architecture encompasses three fundamental search domains:

- **Course Catalog Search:** Enables comprehensive queries across course titles, descriptions, metadata, instructor information, and academic attributes such as credit hours and difficulty levels.
- **People Directory Search:** Facilitates user discovery through ID-based and name-based queries, supporting lookup of Students, Instructors, Administrators, and Teaching Assistants.

- **Project Repository Search:** Provides specialized search capabilities for project-based courses and research topics, with filtering by skill requirements, completion status, and difficulty classifications.

1.3 Integration Architecture

The Search and Filter System maintains tight integration with adjacent LMS subsystems, particularly the Evaluation and Grading modules. This integration enables secure, role-based access to personal Academic Records, ensuring that sensitive grade information is protected while remaining accessible to authorized users through dedicated search pathways.

2. Architectural Design Principles

2.1 Object-Oriented Implementation

The module exemplifies core object-oriented programming principles through strategic application of fundamental design patterns:

- **Encapsulation:** Data structures and search algorithms are bundled within distinct class boundaries, with private data members protected by public accessor methods. The SearchCore class, for instance, encapsulates all keyword-matching logic while exposing only necessary query methods to external clients.
- **Abstraction:** Complex filtering algorithms and search indexing mechanisms are hidden behind intuitive, user-friendly interfaces. End users interact with simple search boxes and dropdown filters, while the underlying implementation leverages sophisticated data structures such as inverted indices and hash tables for optimal performance.
- **Inheritance:** The filter hierarchy implements inheritance through the AbstractFilter base class, which defines common filtering contracts while allowing specialized subclasses (SubjectFilter, LevelFilter, CreditFilter) to provide domain-specific implementations.
- **Polymorphism:** The Strategy Pattern implementation enables runtime selection of filtering algorithms. Client code can invoke the same `applyFilter()` method on any AbstractFilter instance, with the actual filtering behavior determined by the concrete subclass at runtime.

2.2 Design Pattern Implementation

- **Strategy Pattern:** The `AbstractFilter` class and its concrete implementations (`SubjectFilter`, `LevelFilter`) exemplify the Strategy Pattern. This design allows the system to swap filtering algorithms dynamically without modifying client code, supporting extensibility and maintainability.
 - **Comparable Interface:** The `Course` class implements the `CourseSortable` interface (extending Java's `Comparable`), enabling natural ordering of search results based on multiple criteria including course title, credit hours, enrollment statistics, and user-defined preferences.
-

3. Class Architecture and Relationships

3.1 Core Class Hierarchy

The system architecture comprises several interconnected class hierarchies that work cohesively to deliver search and filtering functionality:

- **SearchCore Class:** Serves as the central search engine, implementing keyword-based query processing across all searchable domains. This class maintains references to data repositories (`CourseRepository`, `UserRepository`, `ProjectRepository`) and coordinates result aggregation.
- **AbstractFilter Class:** Defines the abstract filtering contract through the abstract method `applyFilter(courseList: List<Course>)`. This design enforces consistent filtering interfaces across all filter implementations while permitting specialized logic in concrete subclasses.
- **Filter Implementations:**
 - *SubjectFilter*: Filters courses by academic subject categories.
 - *LevelFilter*: Filters courses by difficulty level.
 - *CreditFilter*: Filters courses by credit hour requirements.
 - *InstructorFilter*: Filters courses by assigned instructor name or ID.
- **Course Class:** Represents individual course entities with comprehensive metadata including title, description, credit hours, subject category, level, instructor assignment, enrollment capacity, and scheduling information.
- **User Classes:** Polymorphic user hierarchy including `Student`, `Instructor`, `Administrator`, and `TA` subclasses, each inheriting from a base `User` class.

3.2 Interface Contracts

CourseSortable Interface: Extends the standard `Comparable<Course>` interface to provide course-specific comparison logic. The `compareTo(other: Course)` method implementation supports multiple sorting criteria:

1. Primary sort by course title (alphabetical).
 2. Secondary sort by credit hours (descending).
 3. Tertiary sort by enrollment availability.
 4. Quaternary sort by instructor rating (when available).
-

Hierarchy(nouns) : (Mermaid)

Code:

graph TD

A[Search & Filter System]

A --> B[Search Domains]

A --> C[Core Components]

A --> D[Data Entities]

B --> B1[Course Catalog]

B --> B2[People Directory]

B --> B3[Project Repository]

C --> C1[Keyword Engine]

C --> C2[Filter Set]

C1 --> C1a[SearchCore]

C1 --> C1b[Token Store]

C2 --> C2a[Subject Filter]

C2 --> C2b[Level Filter]

C2 --> C2c[Credit Filter]

C2 --> C2d[Instructor Filter]

D --> D1[Course Data]

D --> D2[User Data]

D --> D3[Project Data]

D1 --> D1a[Title]

D1 --> D1b[Credits]

D1 --> D1c[Difficulty]

D1 --> D1d[Instructor]

D2 --> D2a[Student]

D2 --> D2b[Instructor]

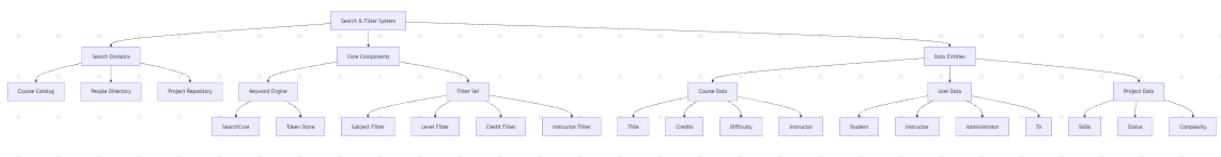
D2 --> D2c[Administrator]

D2 --> D2d[TA]

D3 --> D3a[Skills]

D3 --> D3b[Status]

D3 --> D3c[Complexity]



Hierarchy(verbs) : (Mermaid)

Code:

graph TD

A[System Actions]

A --> B[Search]

B --> B1[Process Query]

B1 --> B2[Match Keywords]

B2 --> B3[Aggregate Results]

A --> C[Filter]

C --> C1[Select Strategy]

C1 --> C2[Apply Filter]

C2 --> C3[Refine Results]

A --> D[Sort]

D --> D1[Compare Items]

D1 --> D2[Order By Criteria]

A --> E[Access Records]

E --> E1[Check Permissions]

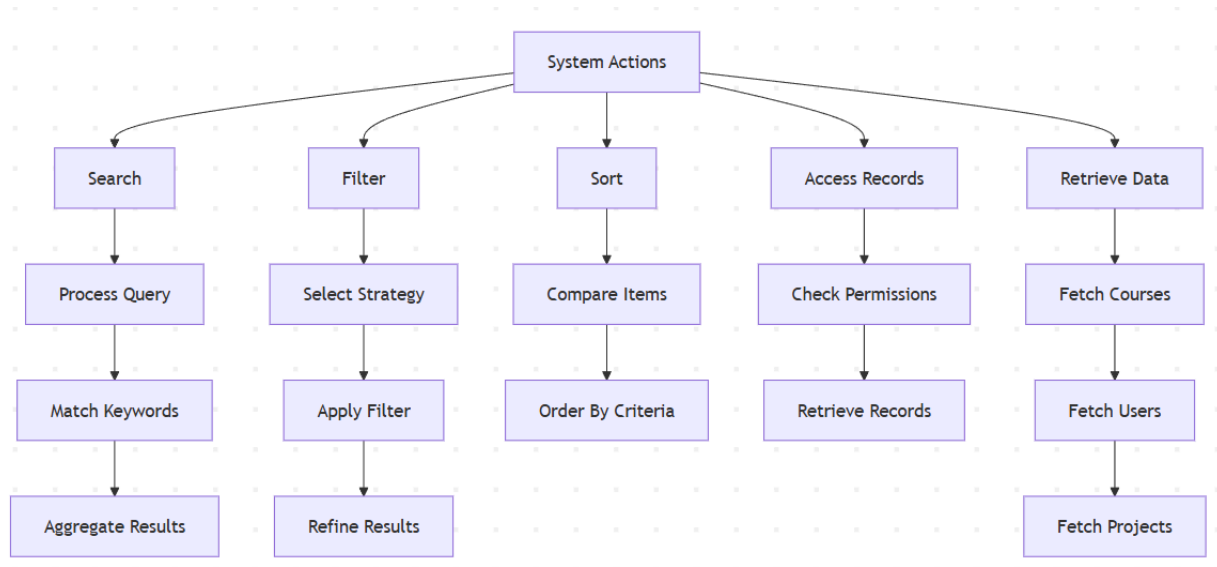
E1 --> E2[Retrieve Records]

A --> F[Retrieve Data]

F --> F1[Fetch Courses]

F1 --> F2[Fetch Users]

F2 --> F3[Fetch Projects]



UML Diagram(Class Diagram):

Code:

```
```mermaid
classDiagram
direction TB
class SearchFilterSystem {
+search(term)
+filter(criteria)
+displayResults()
+refreshUI()
}
SearchFilterSystem --> CourseCatalog : "manages >"
SearchFilterSystem --> PeopleDirectory : "connects to >"
SearchFilterSystem --> ProjectRepository : "manages >"
class CourseCatalog {
+listCourses()
+getCourse(id)
+applyFilters()
+searchCourse(keyword)
+sortBy(criteria)
}
class Course {
```



```
-title : String
-creditHours : int
-subject : String
-level : String
-instructor : String
+getInfo()
+matchesFilter(filter)
+compareBy(field)
}

class Filters {
+apply(course)
+addFilter(filter)
+clearFilters()
+combineFilters()
}

class SubjectFilter {
-subject : String
+apply(course)
}

class LevelFilter {
-level : String
+apply(course)
}

class CreditFilter {
```

-credits : int

+apply(course)

}

class InstructorFilter {

-instructor : String

+apply(course)

}

CourseCatalog --> Course : "contains \*"

CourseCatalog --> Filters : "uses >"

Filters --> SubjectFilter : "includes >"

Filters --> LevelFilter : "includes >"

Filters --> CreditFilter : "includes >"

Filters --> InstructorFilter : "includes >"

class PeopleDirectory {

+searchPerson(name)

+filterByRole(role)

+listAll()

}

class User {

-name : String

-email : String

+login()

+logout()

+getRole()

```
}
```

```
class Student {
```

```
-studentID : String
```

```
+viewCourses()
```

```
+submitAssignments()
```

```
}
```

```
class Instructor {
```

```
-department : String
```

```
+viewTeachingSchedule()
```

```
+assignGrades()
```

```
+approveProjects()
```

```
}
```

```
class Administrator {
```

```
-adminLevel : int
```

```
+manageUsers()
```

```
+updateSystemSettings()
```

```
+generateReports()
```

```
}
```

```
class TA {
```

```
-assignedCourses : List~String~
```

```
+assistCourse()
```

```
+gradeAssignments()
```

```
}
```

```
PeopleDirectory --> User : "stores *"
```

```
User <|-- Student : "inherits <"
User <|-- Instructor : "inherits <"
User <|-- Administrator : "inherits <"
User <|-- TA : "inherits <"

class ProjectRepository {
+searchProjects()
+filterBySkill(skill)
+listAllProjects()
+sortProjects(criteria)
}

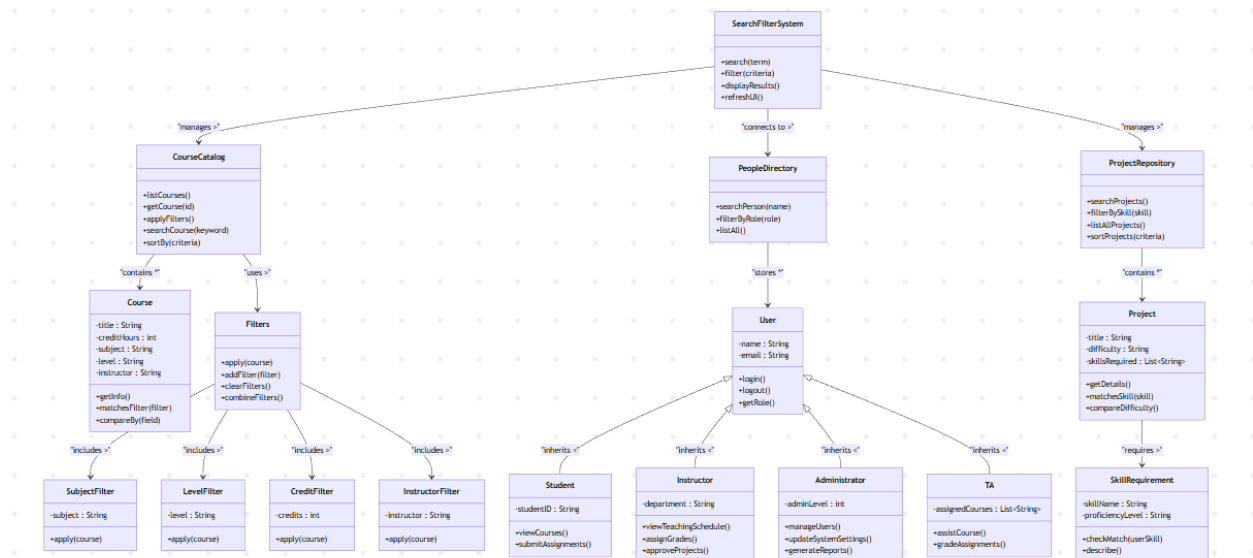
class Project {
-title : String
-difficulty : String
-skillsRequired : List~String~
+getDetails()
+matchesSkill(skill)
+compareDifficulty()
}

class SkillRequirement {
-skillName : String
-proficiencyLevel : String
+checkMatch(userSkill)
+describe()
}
```

ProjectRepository --> Project : "contains \*"

Project --> SkillRequirement : "requires >"

...



## Abstract Class And Comparable Interface UML Diagram:

### Code:

classDiagram

direction TB

class CourseSortable {

<<interface>>

+compareTo(course)

}

class SortStrategy {

<<interface>>

+sort(items)

}

class AbstractFilter {

<<abstract>>

+applyFilter(list)

+getFilterName()

}

class SearchCore {

-courseRepo

-userRepo

-projectRepo

+searchCourses(keyword)

+searchPeople(keyword)

+searchProjects(keyword)

```
+rankResults()
}

class CourseRepository {
+getAllCourses()
+findByKeyword(keyword)
}

class UserRepository {
+findUserByID(id)
+findUserByName(name)
}

class ProjectRepository {
+findProjectBySkill(skill)
+findProjectByDifficulty(level)
}

class Course {
-title
-credits
-subject
-level
-instructor
+compareTo(other)
+getMetadata()
}

class Metadata {
```

-title

-creditHours

-subject

-level

-instructor

+getSummary()

}

Course \*-- Metadata : "has"

Course ..|> CourseSortable : implements

class SubjectFilter {

+applyFilter(list)

+getFilterName()

}

class LevelFilter {

+applyFilter(list)

+getFilterName()

}

class CreditFilter {

+applyFilter(list)

+getFilterName()

}

class InstructorFilter {

+applyFilter(list)

+getFilterName()



```
}
```

```
SubjectFilter --|> AbstractFilter : extends
```

```
LevelFilter --|> AbstractFilter : extends
```

```
CreditFilter --|> AbstractFilter : extends
```

```
InstructorFilter --|> AbstractFilter : extends
```

```
class TitleSort {
```

```
+sort(items)
```

```
}
```

```
class CreditSort {
```

```
+sort(items)
```

```
}
```

```
class DifficultySort {
```

```
+sort(items)
```

```
}
```

```
TitleSort ..|> SortStrategy
```

```
CreditSort ..|> SortStrategy
```

```
DifficultySort ..|> SortStrategy
```

```
SearchCore ..> SortStrategy : "uses strategy"
```

```
class User {
```

```
<<abstract>>
```

```
-id
```

```
-name
```

```
-email
```

```
+login()
```

```
+logout()
}

class Student {
+viewCourses()
+searchPeople()
}

class Instructor {
+searchStudents()
+manageProjects()
}

class Administrator {
+monitorSearches()
+updateRecords()
}

class TA {
+assistInstructor()
+searchCourses()
}

User <|-- Student
User <|-- Instructor
User <|-- Administrator
User <|-- TA

UserRepository *-- User : "stores users"

class Project {
```

-title

-difficulty

-requiredSkills

+getDetails()

}

class SkillRequirement {

-skill

-level

}

class Difficulty {

-value

}

Project \*-- SkillRequirement : "requires"

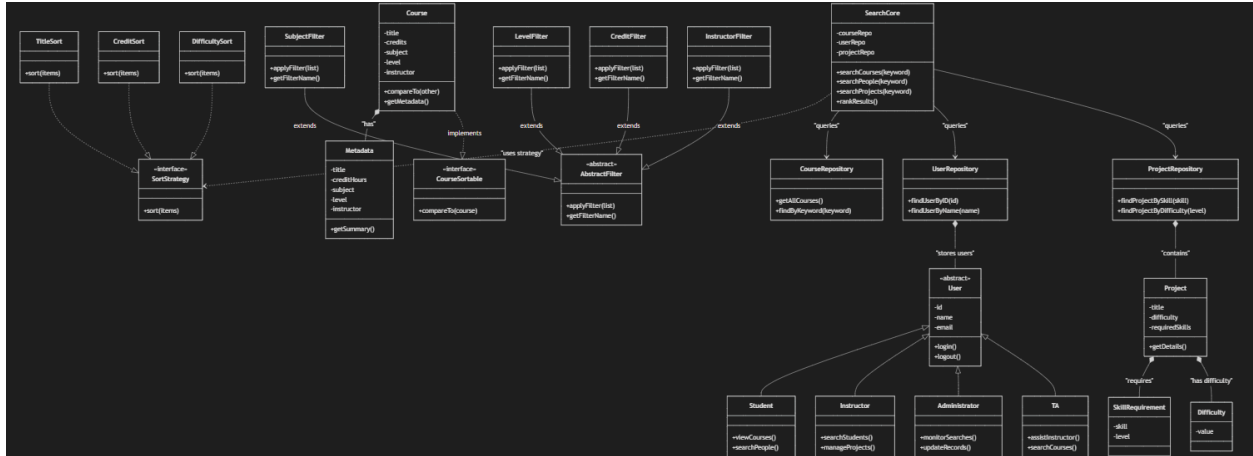
Project \*-- Difficulty : "has difficulty"

ProjectRepository \*-- Project : "contains"

SearchCore --> CourseRepository : "queries"

SearchCore --> UserRepository : "queries"

SearchCore --> ProjectRepository : "queries"



## Abstract Class (AbstractFilter)

- It contains an **abstract method** called `applyFilter(courseList: List<Course>)`. This is a contract. It forces any "child" class that inherits from `AbstractFilter` (like `SubjectFilter` and `LevelFilter`) to write its own specific version of that method (the **Strategy Pattern**).

## Comparable Interface (CourseSortable)

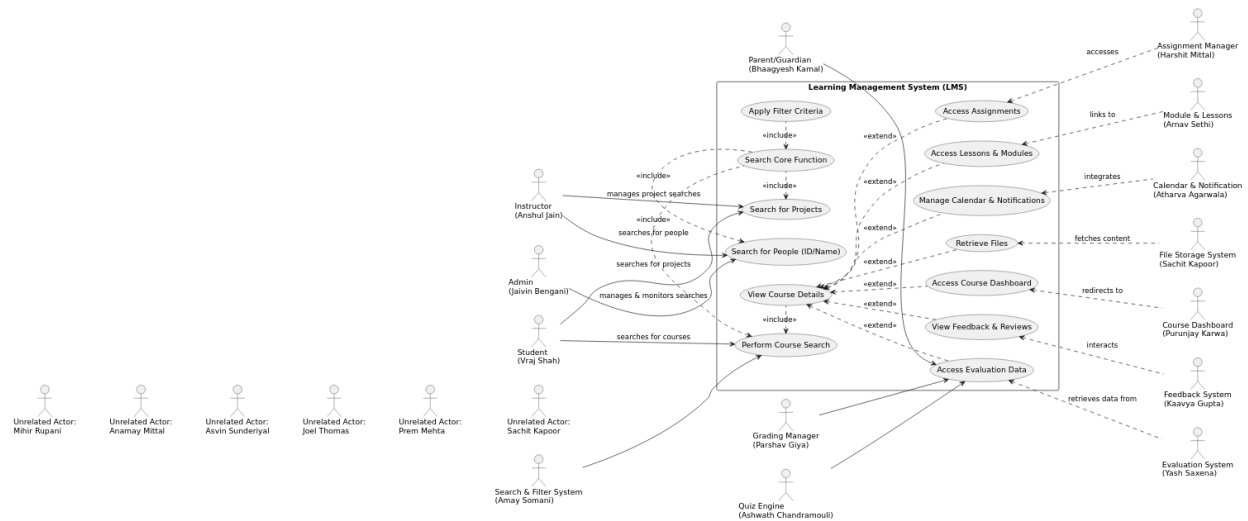
- Your **Course** class implements this interface. This means it is *required* to have a `compareTo(other: Course)` method. Inside this method, it will gather its own data (like **title** and **creditHours**) and determine the course's natural ordering for default sorting capabilities.

Together, the abstract class and interface ensure:

- Reusability and consistency (via abstract class)**
- Flexibility and polymorphic behavior (via interface)**
- Clean separation of shared vs. specialized responsibilities**

This makes the overall design scalable, maintainable, and adaptable to future extensions.

# Use Case Diagram:



## Use Case Specification:

### 1.Perform Course Search:

Step	Action
1	The system displays a search bar and available filter options on the LMS dashboard.
2	The user enters <b>keywords</b> (course title, subject, or instructor name) into the search bar.
3	The system retrieves an initial list of matching courses from the Course Catalog.
4	The system prompts the user with available filters such as subject category, credit hours, and course level.

5	The user applies one or more filters to refine the search results.
6	The system processes the filter criteria and updates the search results dynamically.
7	The user selects a specific course from the refined search results.
8	The system displays detailed information about the selected course (instructor, syllabus, credits, and related modules).

## 2. Search for People (ID/Name):

Step	Action
1	The system displays an input form specifically optimized for searching users (e.g., ID format validation).
2	The user enters the search criteria (e.g., college ID or full name).
3	The system executes the <b>Search Core Function</b> (UC_Search_Core) to find matching profiles.
4	The system displays a list of matching <b>User</b> records (Student, Instructor, etc.).

5	The user selects a specific profile.
6	The system displays the detailed profile information (contact, role, courses).

### 3. Search for Projects:

Step	Action
1	The system displays a search form with filtering options relevant to projects (e.g., "Skill Tag," "Completion Status").
2	The user enters criteria (e.g., project name or required skills).
3	The system executes the <b>Search Core Function</b> (UC_Search_Core) against the Project Catalog.

4	The system displays a list of relevant ProjectCourse results.
5	The user applies additional filters (e.g., "Difficulty: Advanced").
6	The system processes the filter criteria and updates the displayed results dynamically.

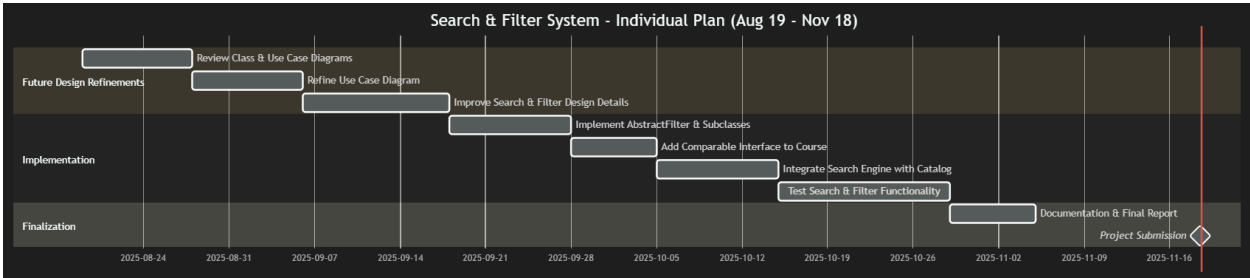
#### 4. Access Academic Records:

Step	Action
1	The system verifies the user's identity and permissions to access the records.
2	The system sends a request to the <b>Evaluation System</b> for the user's grade history.
3	The <b>Evaluation System</b> gathers the required scores and feedback.



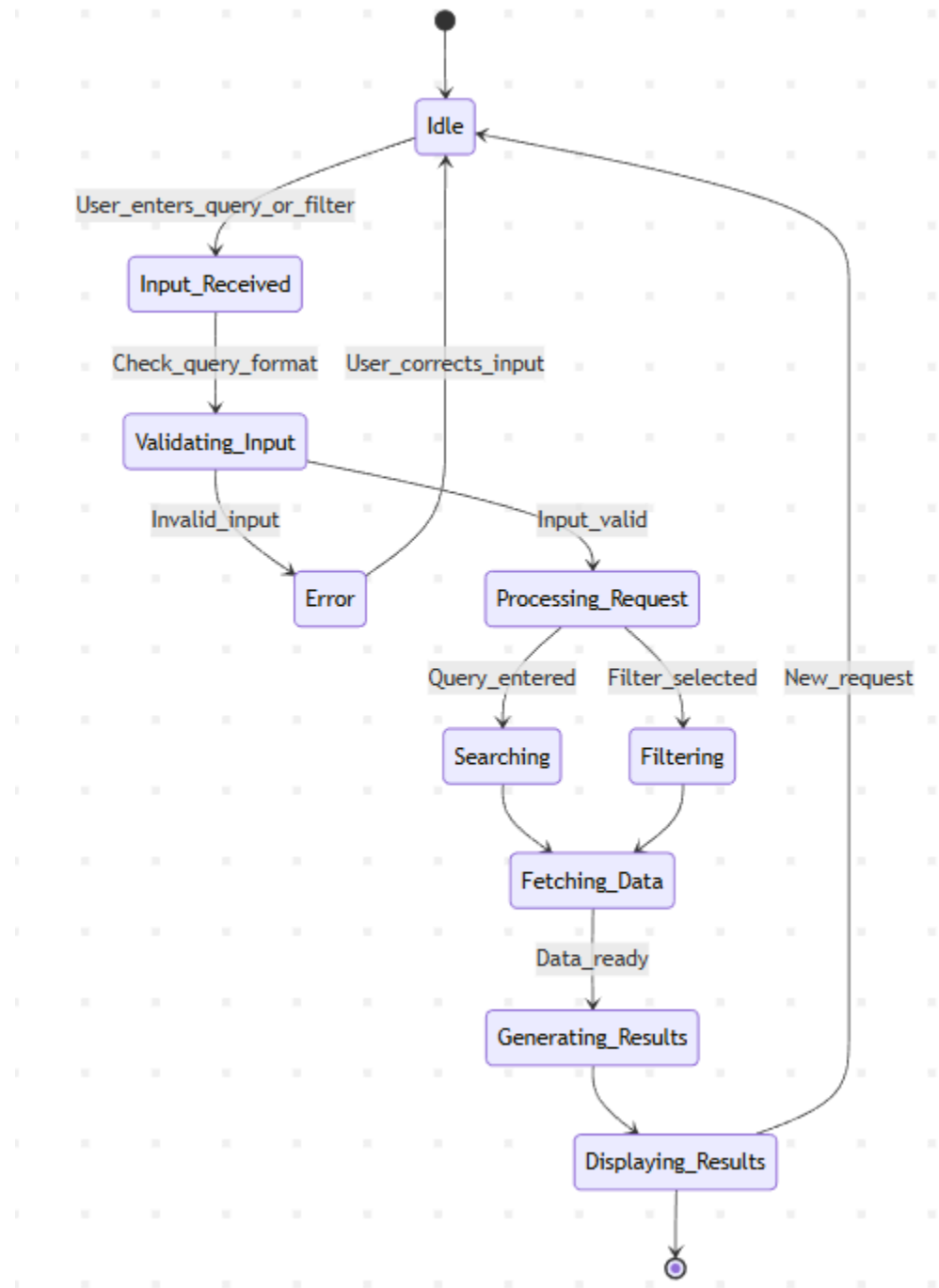
4	The <b>Grading Manager</b> formats the raw data into an official academic report.
5	The system displays the formatted academic record report, categorized by semester and course.
6	The user can choose to export or print the report.

Gantt Chart:

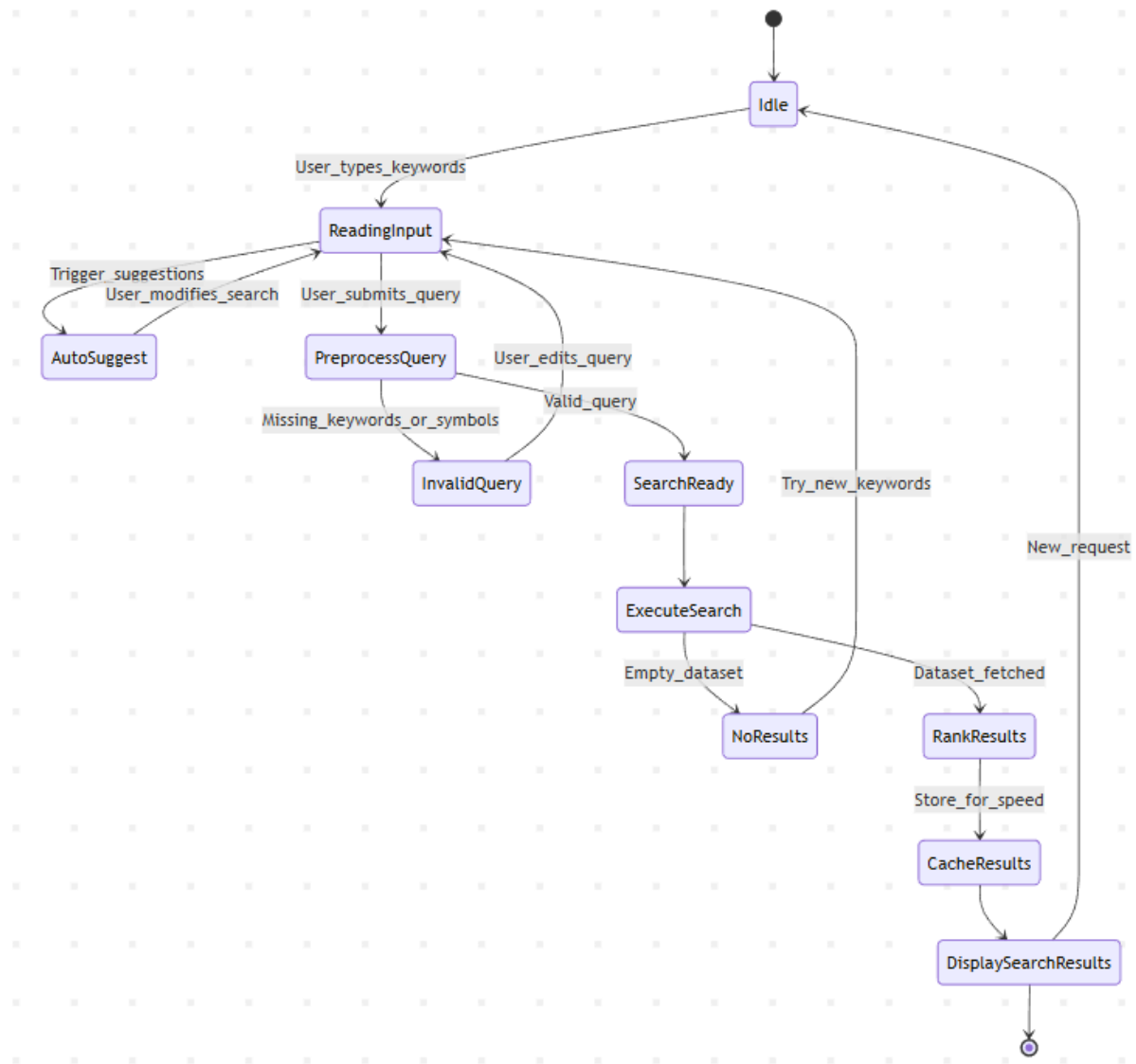


# State Charts:

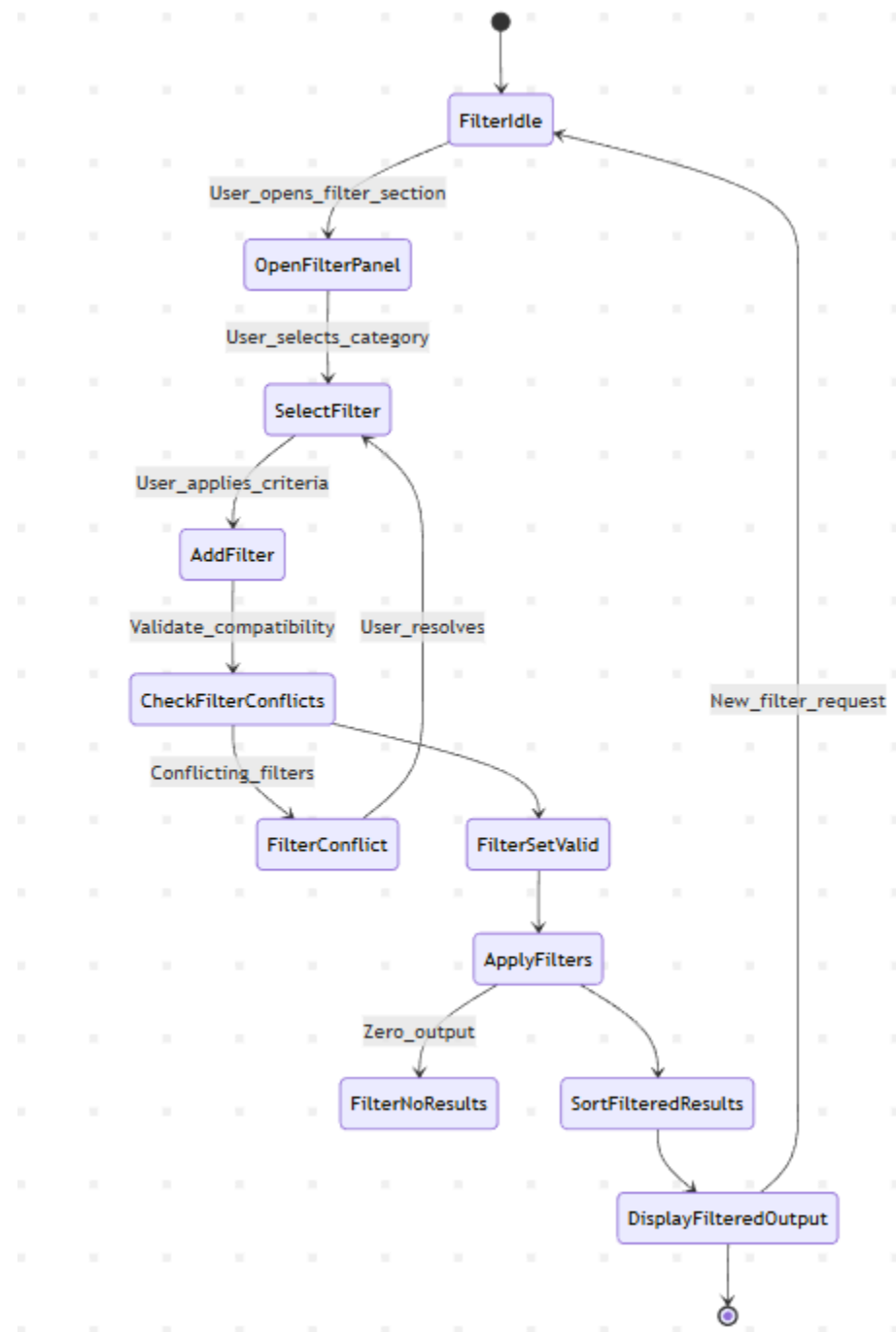
## 1. Project-Level State Chart (Overall System Behaviour)



## 2. Detailed Search Operation State Chart



### 3. Detailed Filter Operation State Chart



## Sequence Diagram:

